P-RFO Notes

Partitioned Rational-Function Optimization for Saddle Search

Notes for ChemDM

A conceptual reference for partitioned rational-function optimization as implemented in `src/chemdm/prfo.py`. Captures the *why*, not just the *what* — in particular why BFGS fails for saddle search, where the "shift" in P-RFO comes from, and why this is a saddle *finder* and not a reaction *path* method.

1 The problem

Find a first-order saddle point of a potential energy surface $E(x)$ using *only forces* $-\nabla E$, no analytic Hessian. The first-order saddle has

$$\nabla E = 0, \qquad H = \nabla^2 E \text{ has exactly one negative eigenvalue.}$$

Two challenges: (a) the gradient alone is zero at every minimum *and* every saddle, so gradient-following cannot distinguish them, and (b) the Hessian is expensive — the very thing we want to avoid computing.

2 Why BFGS specifically fails

The BFGS quasi-Newton update

$$H_{k+1} = H_k + \frac{yy^T}{y^T s} - \frac{H_k s s^T H_k}{s^T H_k s}, \qquad s = x_{k+1} - x_k, \quad y = g_{k+1} - g_k,$$

provably preserves positive definiteness when the curvature condition $y^T s > 0$ holds. Line search enforces this condition for minimization.

The consequence for saddle search is fatal: if $H_0 = I$ (PD), every subsequent H_k is PD by induction. The lowest eigenvalue can never cross zero. The Newton step $-H_k^{-1} g$ always points downhill. **You converge to a minimum, no matter how close you start to a saddle.**

This is not a corner case — it is a fundamental structural mismatch. BFGS was designed for minimization and bakes that intent into its update rule.

The fix is to use a quasi-Newton update that *does not preserve PD*:

Update	PD-preserving	Used for
BFGS	yes	minimization
SR1 (symmetric rank-1)	no	saddle search (unstable if $r^T s$ small)
PSB (Powell-symmetric-Broyden)	no	saddle search (stable, slower)
Bofill (SR1/PSB convex mix)	no	saddle search (best of both)

`prfo.py` uses Bofill with weight $\varphi = (r^T s)^2/(\|r\|^2\|s\|^2)$ — the cosine squared between the residual $r = y - H_k s$ and s. SR1-heavy when those vectors align, PSB-heavy when they do not.

3 Newton with an indefinite Hessian — almost works

Suppose we *had* the true indefinite Hessian at a saddle. Decompose the Newton step $\Delta x = -H^{-1}g$ in the eigenbasis:

$$\Delta x_i = -\frac{g_i}{\lambda_i}.$$

For positive λ_i, Δx_i opposes g_i — descent. For *negative* λ_i, Δx_i aligns with g_i — **ascent**. So Newton with the true Hessian automatically climbs the unstable mode and descends the others. This is eigenvector-following before we even introduce RFO.

But two problems remain:

1. **Step blows up near singular Hessians.** Any $\lambda_i \approx 0$ makes $|\Delta x_i| = |g_i/\lambda_i|$ enormous. We need regularization.
2. **Wrong-mode following.** Early in the search, the "naturally" most negative eigenvalue may not be the one we actually want to ascend (multiple soft modes, eigenvalues crossing zero during the search). We need explicit *control* over which mode to follow.

RFO solves both at once.

4 The RFO step as shift-invert

Replace the quadratic energy model with a **rational function**:

$$\Delta E(\Delta x) \;\approx\; \frac{g^T\Delta x + \frac{1}{2}\,\Delta x^T H \Delta x}{1 + \Delta x^T \Delta x} \;=\; \frac{N(\Delta x)}{D(\Delta x)}.$$

Stationarity by direct differentiation. Setting $\nabla_{\Delta x}(N/D) = 0$ gives $\nabla N \cdot D = N \cdot \nabla D$, i.e. $\nabla N = (N/D)\,\nabla D$. With $\nabla N = g + H\Delta x$ and $\nabla D = 2\Delta x$:

$$g + H\Delta x \;=\; \frac{N(\Delta x)}{D(\Delta x)} \cdot 2\Delta x.$$

The right-hand side carries a scalar that *depends on* Δx — the value of the model at the stationary point. Defining

$$\mu \;:=\; 2\,E_{\text{model}}(\Delta x^\star) \;=\; \frac{2N(\Delta x^\star)}{D(\Delta x^\star)} \tag{1}$$

at any stationary point $\Delta x^\star$, this collapses to

$$(H - \mu I)\,\Delta x \;=\; -g. \tag{2}$$

This is N scalar equations. But the system now has $N+1$ unknowns: the N components of Δx plus the scalar μ. The missing equation is (1) itself — the implicit definition of μ.

The implicit definition simplifies to a clean linear constraint. Rewrite (1) as $\mu(1 + \Delta x^T\Delta x) = 2g^T\Delta x + \Delta x^T H \Delta x$, then use (2) multiplied by Δx^T from the left, $\Delta x^T H\Delta x + g^T\Delta x = \mu\,\Delta x^T\Delta x$, to eliminate the quadratic term:

$$\mu + \mu\,\Delta x^T\Delta x \;=\; 2g^T\Delta x + (\mu\,\Delta x^T\Delta x - g^T\Delta x) \;\Longrightarrow\; \mu \;=\; g^T\Delta x.$$

So the (N+1)-th equation, once cleaned up, is

$$g^T\Delta x \;=\; \mu. \tag{3}$$

Packaging both equations in one $(N{+}1) \times (N{+}1)$ **matrix.** Equations (2) and (3) together form exactly the **augmented eigenvalue problem**:

$$\begin{pmatrix} H & g \\ g^T & 0 \end{pmatrix} \begin{pmatrix} \Delta x \\ 1 \end{pmatrix} \;=\; \mu \begin{pmatrix} \Delta x \\ 1 \end{pmatrix}. \tag{4}$$

The top N rows reproduce (2); the bottom row reproduces (3). Neither row is imposed as a separate constraint — both come from the same stationarity of the rational function, with the bottom row being the simplified form of μ's implicit definition once the top row already holds.

Why this packaging is useful. Eigenvalue problems for symmetric matrices have very clean spectral structure (real eigenvalues, orthogonal eigenvectors, interlacing). A single `eigh` call on the $(N{+}1) \times (N{+}1)$ augmented matrix solves both (2) and (3) simultaneously, returning all $N+1$ valid $(\mu, \Delta x)$ pairs at once. Solving "N nonlinear gradient equations plus an implicit scalar definition" is much messier generically — the augmented formulation buys a clean numerical recipe at the cost of one extra row.

Reading off the step. Once an eigenvalue μ and corresponding eigenvector (v, v_{N+1}) are obtained, normalise so $v_{N+1} = 1$ and read Δx from the first N components. Equivalently, from (2) directly:

$$\boxed{\Delta x = -(H - \mu I)^{-1} g.} \tag{5}$$

That is shift-invert. The Hessian gets shifted by μ, inverted, applied to $-g$. The same operator appears in:

- **Levenberg-Marquardt** for nonlinear least squares (shift = damping α);
- **Trust-region Newton (Moré-Sorensen)** — shift chosen to satisfy $\|\Delta x\| = \text{trust\_radius}$;
- **Tikhonov regularization** for ill-posed inverse problems;
- **Inverse power iteration / shift-invert Lanczos** for eigenvalue problems.

What makes RFO distinctive among shift-invert methods is the **rule for picking** μ: it is an eigenvalue of the augmented $(N+1) \times (N+1)$ matrix in equation (4). Equivalently, the same μ is a root of the scalar **secular equation** derived below.

Deriving the secular equation. Substitute (5) into the bottom-row relation (3):

$$g^T \Delta x \;=\; \mu \quad\Longrightarrow\quad g^T \left[-(H - \mu I)^{-1} g\right] \;=\; \mu \quad\Longrightarrow\quad -\, g^T (H - \mu I)^{-1} g \;=\; \mu.$$

This is already a scalar equation in μ, but it is more useful in the *eigenbasis of* H. Diagonalise $H = U \Lambda U^T$, where $\Lambda = \text{diag}(\lambda_1, \ldots, \lambda_N)$ and the columns of U are the orthonormal eigenvectors. Define

$$f \;:=\; U^T g, \qquad f_i = (\text{eigvec}_i)^T g \quad \text{(the gradient's component along mode } i\text{)}.$$

Then the shifted Hessian inverts diagonally:

$$(H - \mu I)^{-1} \;=\; U(\Lambda - \mu I)^{-1} U^T \;=\; U \,\text{diag}\Big(\tfrac{1}{\lambda_i - \mu}\Big)\, U^T,$$

and the quadratic form becomes a plain sum over modes:

$$g^T(H-\mu I)^{-1}g \;=\; (U^Tg)^T\,\mathrm{diag}\Big(\tfrac{1}{\lambda_i-\mu}\Big)(U^Tg) \;=\; \sum_i \frac{f_i^2}{\lambda_i-\mu}.$$

Substituting back into the scalar equation:

$$-\sum_i \frac{f_i^2}{\lambda_i-\mu} \;=\; \mu \quad\Longleftrightarrow\quad \sum_i \frac{f_i^2}{\mu-\lambda_i} \;=\; \mu.$$

That is the **secular equation**:

$$\boxed{\sum_i \frac{f_i^2}{\mu-\lambda_i} \;=\; \mu.} \tag{6}$$

Why interlacing. The left-hand side is a sum of simple poles at each λ_i with positive residue f_i^2. Between any two consecutive eigenvalues $\lambda_i < \lambda_{i+1}$, the function jumps from $+\infty$ (as $\mu \to \lambda_i^+$) to $-\infty$ (as $\mu \to \lambda_{i+1}^-$), strictly decreasing in between. The right-hand side μ is just a line. A monotone-decreasing function crossing a line on a bounded interval gives exactly one intersection per gap. Plus one extra root below $\lambda_{\min}$ (LHS $\to 0^-$ from below at $-\infty$, line at $-\infty$, so they cross once) and one above $\lambda_{\max}$ (analogous). Total: $N+1$ roots, *interleaved* with the λ_i:

$$\mu_0 \;<\; \lambda_1 \;<\; \mu_1 \;<\; \lambda_2 \;<\; \cdots \;<\; \lambda_N \;<\; \mu_N.$$

These are precisely the $N+1$ eigenvalues of the augmented matrix in (4) (which has dimension $N+1$, so $N+1$ eigenvalues). The two descriptions — "eigenvalue of the augmented matrix" and "root of the secular equation" — are the same set of numbers.

This scalar equation has $N+1$ roots, interleaved with the λ_i's plus one below the lowest and one above the highest eigenvalue. **Each root is a valid shift; each gives a different step type:**

- **Lowest root** $\mu < \lambda_{\min}$: all $(\lambda_i - \mu) > 0$, so the step descends in every mode. Minimization RFO.
- **Largest root** $\mu > \lambda_{\max}$: all $(\lambda_i - \mu) < 0$, so the step ascends in every mode. Not usually what you want.
- **Intermediate roots**: mixed — ascends in some modes, descends in others.

The choice of root encodes the method's intent.

The rational denominator $1+\|\Delta x\|^2$ is what gives RFO its **automatic step bounding**. As $\|g\|$ grows or H becomes singular, μ grows in magnitude so $(\lambda_i - \mu)$ stays bounded away from zero. The step Δx cannot blow up by construction — no trust-region constraint required. (We still impose one as a second safety net in `prfo.py`, used adaptively via the $\rho = \Delta E_{\text{actual}}/\Delta E_{\text{predicted}}$ ratio.)

5 The partitioning (the "P" in P-RFO)

For transition-state search we want one mode to ascend and the rest to descend.

Where the subproblems come from. Rewrite the augmented eigenvalue problem (4) in the *eigenbasis of* H. Let $H = U\Lambda U^T$ with eigenvalues λ_i, and let $\Delta q = U^T \Delta x$ be the step in the eigenbasis, $f_i = (U^T g)_i$ the gradient component along mode i. Conjugating by U (i.e. multiplying on each side by $\mathrm{diag}(U, 1)$) diagonalises the upper block of the augmented matrix:

$$\underbrace{\begin{pmatrix} \lambda_1 & & & f_1 \\ & \ddots & & \vdots \\ & & \lambda_N & f_N \\ f_1 & \cdots & f_N & 0 \end{pmatrix}}_{(N+1)\times(N+1)} \begin{pmatrix} \Delta q \\ 1 \end{pmatrix} \;=\; \mu \begin{pmatrix} \Delta q \\ 1 \end{pmatrix}. \tag{7}$$

The same eigenvalues, same eigenvectors — just rotated into a basis where the coupling structure is transparent. Now look at any single row i in the top block:

$$(\lambda_i - \mu)\,\Delta q_i \;=\; -f_i, \quad \text{i.e.} \quad \Delta q_i \;=\; -\frac{f_i}{\lambda_i - \mu}. \tag{8}$$

Each mode's step depends only on its own (λ_i, f_i) and the shared scalar μ. The modes do not couple to each other; they couple only through the common shift.

For minimization, one chooses $\mu = \mu_0 < \lambda_{\min}$ in the full secular equation (6) so $(\lambda_i - \mu_0) > 0$ for every i — every mode descends. But for transition-state search, we want *different signs for different modes*: ascent in the chosen mode k, descent in all others. No single root of (6) can do that unless mode k happens to be the softest. The partitioning trick is to **use a different shift on each subspace**, computed from a smaller augmented eigenvalue problem on the modes of that subspace alone.

Followed (ascent) mode k. Restrict to mode k only: just one λ, one f. The augmented eigenvalue problem (7) collapses to a 2×2 block:

$$\begin{pmatrix} \lambda_k & f_k \\ f_k & 0 \end{pmatrix} \begin{pmatrix} v \\ 1 \end{pmatrix} \;=\; \mu_+ \begin{pmatrix} v \\ 1 \end{pmatrix}. \tag{9}$$

Its characteristic polynomial is $(\lambda_k - \mu)(-\mu) - f_k^2 = 0$, i.e. $\mu^2 - \lambda_k \mu - f_k^2 = 0$, with two roots:

$$\mu_\pm \;=\; \tfrac{1}{2}\left(\lambda_k \pm \sqrt{\lambda_k^2 + 4f_k^2}\right).$$

Since $\sqrt{\lambda_k^2 + 4f_k^2} > |\lambda_k|$, we have $\mu_+ > \lambda_k$ and $\mu_- < 0$. Take μ_+. Then $(\lambda_k - \mu_+) < 0$, so

$$\Delta q_k \;=\; -\frac{f_k}{\lambda_k - \mu_+} \;=\; \frac{f_k}{\mu_+ - \lambda_k}$$

points along $+f_k$ — *the same direction as the gradient component in mode k, i.e. uphill.* That is the ascent step.

Descent subspace (the other $N - 1$ modes). Restrict to those modes: drop row k and column k from (7) (leaving an $M \times M$ block with $M = N - 1$ active modes, plus the bottom row and last column of f's). The result is an $(M + 1) \times (M + 1)$ augmented eigenvalue problem on the descent subspace. Its smallest root μ_- lies below $\min_{j \neq k} \lambda_j$, so $(\lambda_j - \mu_-) > 0$ for every remaining mode and the step $\Delta q_j = -f_j/(\lambda_j - \mu_-)$ descends in each.

Why the partitioning is mathematically clean. Both subproblems are themselves *exact RFO problems on subspaces of H*. They inherit every property of the full augmented eigenvalue problem (interlacing roots, shift-invert structure, implicit step bound from the rational-function model), just applied to a smaller piece. The trick is that the eigenbasis decouples the modes' step formulas (8), so a sub-RFO on mode k alone gives a self-consistent shift for that mode, independent of what shift the rest of the modes use. Two RFO subproblems on complementary subspaces of H, each generating its own shift, are **not an approximation** — they are an exact reformulation that allows independent ascent/descent choices.

Reassemble Δq in the eigenbasis and rotate back to get Δx. **One ascent mode, all-others descent.** That is the P in P-RFO.

Which mode k to follow

The partitioning above assumes we have already *chosen* a mode k to ascend. In practice, mode choice is decided iteration-by-iteration based on where you are in configuration space. Three regimes show up.

Iteration 1: at or near a minimum. Every eigenvalue λ_i is positive (or close to it). There is no "unstable mode" yet — you have to *create* one for P-RFO to climb. Standard recipes, in increasing order of effort and quality:

- **Lowest eigenvalue (Cerjan-Miller "gentlest ascent").** Pick $k = \arg\min_i \lambda_i$. Assumes the softest mode at the minimum leads to the chemically relevant saddle — usually true for simple bond-rotation or bond-breaking reactions in floppy molecules, but *not* guaranteed. When a minimum has several saddles adjacent to it, the softest mode points toward one of them, not necessarily the one of interest.
- **Chemical intuition.** Hand-supply a direction u pointing along the expected reaction coordinate (e.g. "the H atom should bend out of the molecular axis"). P-RFO follows the eigenmode with maximum overlap to u. This is what `init_mode` does in `prfo.py` — it adds a rank-1 correction $-2uu^T$ to the initial Hessian so the eigendecomposition has u as a negative-curvature mode from step 1.
- **Lindh-seeded Lanczos (`estimate_lowest_mode`).** Best automatic choice when you have no chemical hint. Lindh's empirical model Hessian seeds Lanczos iteration, which extracts the true lowest mode of the actual Hessian by finite-difference gradient probes — no u guess required.

Subsequent iterations: mode following by maximum overlap. Once mode k has been picked at iteration 1, at every later iteration the Hessian has changed slightly (Bofill update from new gradient information), so its eigenvectors $v_1^{(t)}, \ldots, v_N^{(t)}$ have rotated. Re-picking "the lowest eigenvalue" at every iteration would risk hopping to a *different* mode mid-climb. Baker's 1986 fix: at iteration t, pick

$$k^{(t)} \;=\; \arg\max_i \left|(v_i^{(t)})^T\, u^{(t-1)}\right|,$$

where $u^{(t-1)}$ is the eigenvector followed at iteration $t-1$. **The followed mode is the one most parallel to last step's followed mode.** Justification: as the optimizer moves through configuration space, the true unstable eigenvector varies continuously, so consecutive iterations should track the same physical mode by overlap.

Approaching the saddle. Near the saddle, the followed eigenvalue λ_k approaches zero from above and crosses to negative. The eigenvector stabilises. By the time the optimizer converges, λ_k is the saddle's true unstable eigenvalue (one negative number), and the eigenvector is the saddle's unstable mode — exactly the seed for an IRC computation.

When mode-following breaks. Three failure modes worth knowing:

- **Eigenvalue crossings.** When λ_k and λ_{k+1} nearly cross between iterations, both eigenvectors can rotate rapidly, and maximum-overlap can swap modes. The HCN demo's iterations 19–31 show exactly this: $\lambda_{\text{followed}}$ crossed zero, overlap with the previous followed vector dropped below 0.5, and the optimizer briefly tracked a different mode before re-aligning.
- **True degeneracies (symmetry).** If $\lambda_k = \lambda_j$ for some $j \neq k$ at a symmetry point, the eigenvectors are not unique — `eigh` returns an arbitrary basis of the two-dimensional eigenspace. Overlap-matching becomes ill-defined. HCN's bend at the linear minimum is doubly degenerate (bend-in-y and bend-in-z have the same eigenvalue), which is why our Lanczos-seeded `init_mode` returned a superposition rather than aligned with the eventual TS plane.
- **Wrong starting choice.** If you followed the wrong mode at iteration 1, you converge to a different saddle than intended — or to a higher-order saddle, or wander off. IRC verification catches this: if the endpoints don't match the expected reactant/product, the saddle is real but unrelated.

Robustness improvements (not in current `prfo.py`).

- **Require monotone eigenvalue** when mode-following. Refuse mode switches that swap to an eigenvector with very different λ from the previous followed one. Catches the iter-27 jump in the HCN demo.
- **Recompute the lowest mode by Lanczos at each step.** Drops the dependence on Bofill's Hessian estimate for mode identity. This is what *ART-nouveau* does. Cost: ~ 5 extra gradient calls per step; benefit: the algorithm cannot lose the unstable mode by accident.
- **Track multiple candidate modes.** Run several P-RFO threads in parallel, each following a different initial mode; IRC-verify each at convergence and keep the chemically meaningful one. This is the saddle-enumeration approach.

6 Coordinate systems are orthogonal to RFO

RFO is just linear algebra on (H, g). It works in any coordinate system. What changes between coordinate systems is the *plumbing around* RFO, not RFO itself.

Coords	Dim	Zero modes	Back-transform
Cartesian + trans/rot projection	$3N$	6 (project out)	none
Non-redundant internals	$3N - 6$	0	iterative Wilson B
Redundant internals (Pulay)	$M > 3N - 6$	$M - (3N - 6)$	Wilson B + pseudoinverse

The **three-axis design space** for any TS optimizer is:

Step generator	{Newton, trust-region Newton, RFO/P-RFO, dimer rotation}
Hessian model	{analytic, Bofill, SR1, PSB, BFGS (minimization only)}
Coordinate system	{Cartesian + projection, internal, redundant internal}

Most published "methods" are just specific points in this cube. Our `prfo.py` is {P-RFO, Bofill, Cartesian + projection}. Gaussian's TS optimizer is {P-RFO, Bofill, redundant internals}. Sella is roughly {trust-region Newton, exact Hessian via finite differences, Cartesian + projection}. Dimer codes are {dimer rotation step, no stored Hessian, Cartesian}.

These choices are independent — you can swap any axis without changing the others.

7 P-RFO finds *points*, not *paths*

The optimizer's iterates trace a discrete trajectory through configuration space, but **that trajectory has no chemical meaning.** It depends on initial guess, Hessian model, trust radius, mode-following heuristic — none of which are properties of the molecule. Two runs from different starts can take very different optimizer paths to the same saddle.

The "eigenvector-following" name refers to following an *eigenvector at each iteration*, not to following a *curve through configuration space*. These are different objects.

The reaction path is a separate computation, defined intrinsically by the PES:

IRC (Intrinsic Reaction Coordinate, Fukui) From the converged saddle, integrate $\mathrm{d}x/\mathrm{d}t = -g(x)$ in mass-weighted Cartesians, $\pm$ along the unstable mode. One direction reaches the reactant minimum, the other the product minimum. This curve is geometry-defined; different optimizers that find the same TS yield the same IRC.

This cleanly separates concerns:

Step	Method	Object computed	Cost
1	P-RFO + Bofill	Saddle point (single geometry)	∼20–100 force calls
2	IRC (steepest descent $\pm$)	Reaction path (a curve)	∼50–200 force calls

Both are needed for a complete reaction characterization. P-RFO alone gives you a saddle that might connect anywhere; IRC without a converged TS has no starting point.

8 A note on gradient extremals (and why we do not build them)

Gradient extremal curves (Hoffman, Nord, Ruedenberg) are an alternative class of methods that *do* try to trace a curve directly from the reactant basin. A gradient extremal is the locus of points where ∇E is parallel to a specific Hessian eigenvector.

There is a connection: the discrete trajectory of P-RFO with mode-following is approximately a gradient extremal of the followed mode. So in a loose sense, P-RFO is discretely walking *along* a gradient extremal toward its endpoint at the saddle.

Gradient extremal *following* methods (continuous ODE integration of the curve, adaptive parametrization, branch tracking) make the *curve itself* the object of interest — you walk from the reactant minimum along the curve, through any saddles you encounter, and into the product basin. This is elegant and chemically motivated, but practically harder than the "find TS then IRC" approach for several reasons:

- Gradient extremals branch and recombine — choosing the right branch at every bifurcation requires bookkeeping that IRC does not.
- They can leave the chemically relevant region of the PES, especially in high-dimensional systems.
- They do not necessarily pass through every saddle, and a saddle they do not pass through is invisible to the method.
- Numerically: ODE integration with branch tracking is heavier than "find a stationary point, then steepest-descent."

Our practical choice is the simpler decomposition:

relaxed reactant $\rightarrow$ `estimate_lowest_mode` (Lindh + Lanczos) $\rightarrow$ perturb + P-RFO $\rightarrow$ IRC.

The IRC step is cheap enough (gradient descent, $\sim$150 lines of code) that we do not gain anything from trying to construct the reaction path directly during the saddle search. The gradient-extremal connection is a real mathematical fact, but it does not pay off in code.

9 Summary

P-RFO is **shifted Newton with self-determined shifts**, where the shifts come from an augmented eigenvalue problem and partition configuration space into one ascent mode and a descent subspace. The Hessian is approximated by Bofill (an indefinite-capable quasi-Newton update — BFGS does not work here). The step is implicitly bounded by the rational-function model's denominator; we add an explicit trust radius as a second safety net with adaptive shrink/grow.

This finds a single saddle point per run. To trace the reaction path that connects the saddle to its endpoints, run IRC from the converged TS — a separate, cheap, well-defined computation.

For posterity: the three-axis design space (step generator $\times$ Hessian model $\times$ coordinate system) is what makes the TS-optimization literature navigable. Once you see it that way, "method X vs. method Y" is mostly a question of which point in the cube each method occupies.

10 Postscript: minimization vs. TS is one root choice

The cleanest possible statement of what RFO does: **minimization and transition-state search differ only by which root of the same secular equation you pick.** Everything else — Hessian update, Bofill, trust radius, coordinate system — is shared infrastructure.

Recall the secular equation (6):

$$\sum_i \frac{f_i^2}{\mu - \lambda_i} = \mu.$$

By the interlacing property of its $N + 1$ roots, they sit between the Hessian eigenvalues plus one outside on each end:

$$\mu_0 < \lambda_1 < \mu_1 < \lambda_2 < \mu_2 < \lambda_3 < \cdots < \lambda_N < \mu_N.$$

Each choice of root gives a step of the form $\Delta x = -(H - \mu I)^{-1} g$. What changes is the *sign pattern* of $(\lambda_i - \mu)$ across modes:

Root chosen	Sign pattern of $(\lambda_i - \mu)$	Step type
$\mu_0 < \lambda_1$	all positive	minimization
$\mu_1 \in (\lambda_1, \lambda_2)$	one negative, rest positive	TS, follow mode 1
$\mu_2 \in (\lambda_2, \lambda_3)$	two negative, rest positive	2nd-order saddle search

So, to answer the practical question: **starting from a local minimum, pick $\mu = \mu_1$ instead of μ_0 and the optimizer automatically pursues a transition-state search along the softest mode.** No fake-negative-eigenvalue initialization required, no special-case code, no different update rule. Same Hessian, same gradient, same secular equation — different root.

Two practical caveats:

1. At a true local minimum, $\|g\| = 0$ exactly, so all $f_i = 0$ and the secular equation degenerates: every $\mu_i = \lambda_i$, every step is zero. You must perturb slightly off the minimum to give the optimizer something to climb (and to break mode degeneracy in symmetric systems). This is what the demo's `x_start = x_min + kick * u` step does.

2. The two-shift formulation in Section 5 (μ_+ from a 2×2 subproblem on the followed mode, μ_- from an $M \times M$ subproblem on the rest) is mathematically equivalent to picking μ_1 here. The partitioned form is what production codes implement for numerical reasons (smaller eigenproblems, cleaner mode-following control), but the underlying mathematical content is "pick the secular root that sits above the eigenvalue you want to ascend."

This is the unifying picture: P-RFO and ordinary RFO minimization are the *same algorithm* running on the *same data structure*, choosing a different root of the same scalar equation. When you see it that way, the whole machinery of TS search collapses into a one-line modification of an ordinary minimizer.

References

Schlegel, *WIREs Comput. Mol. Sci.* **1**, 790 (2011). Best tutorial review.

Banerjee, Adams, Simons, Shepard, *J. Phys. Chem.* **89**, 52 (1985). The original P-RFO paper.

Baker, *J. Comput. Chem.* **7**, 385 (1986). The practical recipe (mode following by max overlap) most QC codes follow.

Bofill, *J. Comput. Chem.* **15**, 1 (1994). The Bofill update.

Besalú & Bofill, *Theor. Chem. Acc.* **100**, 265 (1998). Proper trust-radius treatment for RFO.

Wales, *Energy Landscapes* (Cambridge, 2003), ch. 6–7. Wide-angle view connecting RFO to dimer, GAD, and gradient extremals.

Hoffman, Nord, Ruedenberg, *Theor. Chim. Acta* **69**, 265 (1986). Gradient extremal curves (for reference; we do not implement these).